Document number	P2750R0
Date	2022-12-19
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

C Dangling Reduction

Table of contents

Changelog

Abstract

Dangling of the stack is a programming language and specification defect. Even though the programmer does tell the compiler what to create, size and alignment, also approximately where to create an instance, ultimately it is the compiler that does the actual pushing and popping. Further, the specification states when instances are destroyed and if that allows dangling than the specification needs to take responsibility for its decisions. Even if you don't agree with these sentiments, perhaps you can at least acknowledge that their is the perception of defectiveness and consequently this affects whether `c` is used or another language which doesn't have the perceived defect. This proposal considers multiple non breaking changes that can collectively greatly reduce the dangling of the stack.

Motivation

There are multiple resolutions to dangling in the `c` programming language.

1. Produce an error
2. Fix by giving a little more life i.e. `variable scope`
3. Fix by giving a lot more life i.e. `global scope`

All are valid resolutions and individually are better than the others, given the scenario.

Dangling the stack is shocking because is violates our trust in our compilers and language, since they are primarily responsible for the stack.

Produce an error

Some code is just wrong and the compiler should know. As such it would be ideal if the compiler would tell programmers of dangling errors instead of allowing the programmer to proceed forward. Consider some examples:

```
int* f()
{
    return & 1;// dangling
}
```

```
int* f()
{
    int local = 1;
    return &local;// dangling
}
```

In both cases, the programmer is returning a pointer to a local. This is never correct. Consequently, the compiler should not be silent, nor produce a warning but instead should produce an error.

In these cases, these are the facts that a compiler already knows.

- function `f` returns a pointer
- the variable `local` is locally scoped
- function `f` is **directly** returning a pointer to a local

The compiler has all that it needs to report this dangling. It also doesn't need to do whole translation unit analysis or whole program analysis just for this function. All of knowledge needed to perform the analysis on function `f` is available in function `f` meaning dangling detection can occur in parallel for speed, serially for resource utilization or some combination of the two. Since the graph of a function is smaller than the graph of a translation unit or program than the processing time is also minimized as graph algorithms processing time growths quadratically or exponentially based on the number nodes in the graph.

Why perform dangling error detection for even such a trivial example?

- It is embarrassing not to.
- It is the right thing to do since the language, specification and compiler are primarily responsible for the stack.
- It makes the compiler, the teacher.

Even if the language/specification/compiler doesn't handle **indirect** dangling of the stack due to increased resource consumption, even these simple **direct** resolutions are of benefit because teachers can show the compiler reporting an error concerning dangling and from their, move the conversation to more complicated examples that the compiler doesn't handle. It jump starts the conversation. For self taught programmers, as many are, the compiler is the teacher and at least points the programmer in the direction where one should continue their research.

I look at reporting errors as being provided by the standard in three phases.

- Produce errors for simple **direct** dangling
- Produce errors for as much **indirect** dangling
- Allow programmers to contribute information needed for even more **indirect** dangling

Produce errors for simple direct dangling

This example is similar to the previous.

```
struct Point
{
    int x;
    int y;
};

Point* f()
{
    Point local = {1, 3};
    return &local;// dangling
}
```

Basic guards are also needed in the body of a function. Just as the pointer or reference to a local should not be returned because it exits the scope of the lifetime of the local, this should also not occur in the body of any given function.

```
void h(bool b, int i) {
  int* p = nullptr;
  if (b) {
    static int s = 0;
    p = &s;  // OK
  } else {
    int i = 0;
    p = &i;  // error: instance `i` dies before pointer `p`
  }
  // ...
}
```

In order to address these types of dangling, in the language, we need to add a rule into the standard.

RULE: You can not directly assign the address of an instance to a pointer or a reference if the instance dies before the pointer or reference dies.

At worse, this is dangling. At best, this is a logic error.

Produce errors for as much indirect dangling

In this example, the member of a local is still local.

```
struct Point
{
    int x;
    int y;
};

int* f()
{
    Point local = {1, 3};
    return &local.y;// dangling
}
```

This example is indirect because the programmer wrote superfluous code, `p`, which is a pointer to a local.

```
struct Point
{
    int x;
    int y;
};

Point* f()
{
    Point local = {1, 3};
    Point* p = &local;
    return p;// dangling
}
```

Allow programmers to contribute information needed for more indirect dangling

In the following example, a programmer uses an attribute `parameter_dependency` to tell the compiler that the return parameter/argument is dependent upon the `point` parameter/argument. In this call instance, `local` is locally scoped so the returned pointer is to something locally scoped.

```
struct Point
{
    int x;
    int y;
};

[[parameter_dependency(dependent{"return"}, providers{"point"})]]
Point* obfuscating_f(Point* point)
{
    return point;
}

Point* f()
{
    Point local = {1, 3};
    return obfuscating_f(&local);// dangling
}
```

A little more life please

Not all dangling should produce errors. Some code makes perfect sense but based on the current language rules dangle. If we give these instances more life than the code can remain simple and dangling is fixed automatically in the language in a logical way with no intervention from programmers.

values	pointers with `C99` &

`int i = 5;`
`if(whatever)`
`{`
`  i = 7;`
`}`
`else`
`{`
`  i = 9;`
`}`
`// use i`
	`int* i = &5;// or uninitialized`
	`if(whatever)`
	`{`
	`  i = &7;`
	`}`
	`else`
	`{`
	`  i = &9;`
	`}`
	`// use i`

In the `values` example, there is no dangling. Programmers trust the compiler to allocate and deallocate instances on the stack. They have to because the programmer has little to no control over deallocation. With the current `C99` block scope rule, the `pointers` example dangle. In other words, the compilers who are primarily responsible for the stack has rules that needlessly causes dangling. This violates the programmer's trust in their compiler. Variable scope is better because it restores the programmer's trust in their compiler/language by causing [compound] literals to match the value semantics of variables. Further, it avoids dangling throughout the body of the function whether it is anything that introduces new blocks/scopes be that `if`, `switch`, `while`, `for` statements and the nesting of these constructs.

Here is the current C verbiage.

> `2021/10/18 Meneide, C Working Draft` [1]
>
> *"6.5.2.5 Compound literals"*
>
> **paragraph 5**
>
> *"The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has* ***automatic storage duration associated with the enclosing block****."*

What is variable scope? This is what is proposed.

> 2021/10/18 Meneide, C Working Draft [1:1]
>
> *“6.5.2.5 Compound literals”*
>
> **paragraph 5**
>
> *“The value of the compound literal is that of an unnamed object initialized by the initializer list. If the compound literal occurs outside the body of a function, the object has static storage duration; otherwise, it has automatic storage duration associated with the enclosing block or the enclosing block of the variable to which the [compound] literal is assigned to, whichever is greater lifetime.”*

A lot more life please

While the preceding fixes would handle most dangling of the stack in `c` some instances would be better served if it had a lot more life. In particular, if they had `static storage duration`. This would apply for anything that reasonably could be made a constant either implicitly of explicitly.

I apologize for the next reference but I couldn’t say it better or more succinctly.

> ## const type qualifier
>
> …
>
> Objects declared with const-qualified types **MAY** be placed in read-only memory by the compiler, and if the address of a const object is never taken in a program, it may not be stored at all. [2]

Instances that are placed in read-only memory do not dangle because they are global. Instances that are not “stored at all”, because a global/local inline assembly constant was used, does not have anything to dangle. Even a instance that has static storage duration [and const] do not dangle. The issue is right now there are local constants that dangle, according to the standard, but doesn’t dangle because the compiler handled it but the programmer is unaware that it was fixed. Due to this ambiguity, programmer have to pessimistically make their code ugly by adding more superfluous lines of code and more superfluous naming to ensure that dangling does not occur. If **MAY** is changed to a definitive **ARE** than dangling can be fixed in the language in the best possible way for these instances with no programmer intervention needed. Keep in mind too that `const` predates `constexpr` so their are many more instances

that would benefit from this type of dangling resolution. We would just be standardising existing practice. So thet's consider some examples. These are just `const` versions of many of the previous examples.

```
const int* f()
{
    return & 1;// no dangling, logically global constant
}
```

```
const int* f()
{
    const int local = 1;// it could be argued this const is implicit
    return &local;// no dangling, logically global constant
}
```

```
struct Point
{
    int x;
    int y;
};

const Point* f()
{
    const Point local = {1, 3};// it could be argued this const is implicit
    return &local;// no dangling, logically global constant
}
```

```
struct Point
{
    int x;
    int y;
};

const int* f()
{
    const Point local = {1, 3};// it could be argued this const is implicit
    return &local.y;// no dangling, logically global constant
}
```

```
struct Point
{
    int x;
    int y;
};

const Point* f()
{
    const Point global = {1, 3};// it could be argued this const is implicit
    Point* p = &global;
    return p;// no dangling, logically global constant
}



struct Point
{
    int x;
    int y;
};

[[parameter_dependency(dependent{"return"}, providers{"point"})]]
const Point* obfuscating_f(Point* point)
{
    return point;
}

Point* f()
{
    const Point global = {1, 3};// it could be argued this const is implicit
    return obfuscating_f(&global);// no dangling, logically global constant
}
```

```
struct Point
{
    int x;
    int y;
};

[[parameter_dependency(dependent{"return"}, providers{"point"})]]
const Point* obfuscating_f(Point* point)
{
    return point;
}

Point* f()
{
    return obfuscating_f(constexpr &(Point){1, 3});// no dangling, logically globa
}
```

Summary

The advantages to `C++` with adopting this proposal is manifold.

- Safer
  - Greatly reduces dangling of the stack
- Simpler
  - Encourages the use of [compound] literals
  - Encourages the use of `const` and `constexpr`
  - Standardize existing practice allows programmers to take advantage of what compilers have already been doing for a long time

Frequently Asked Questions

Why is this a `C` proposal and not a `C++` proposal?

1. Think of this as a meta-proposal that the `C++` community can offer to the `C` community in order to strengthen our shared community.
2. This paper is a consolidation of multiple dangling papers to show what could be done for a `C` subset of `C++` for code that is more pointer heavy instead of lvalue reference. This scenario may occur in older and larger code bases. Further, this serves to highlight that changes meant to make higher level code safer also applies to lower level.

References

1. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2731.pdf) ↩ ↩

2. https://en.cppreference.com/w/c/language/const (https://en.cppreference.com/w/c/language/const) ↩